

DHARĀ — Earth Change Intelligence

Complete Strategy, Month-Long Roadmap, and AI Execution Prompt Library
— NASA Space Apps Challenge 2026

Prepared for: NISAR Surface Change Challenge Team

September 2026

Contents

Before You Start	3
Part 1 — The Idea, Restated Once, Precisely	4
Why this wins, based on real evidence	4
Award targeting — final and disciplined	4
Part 2 — The Science, in Plain Terms	6
NISAR products used	6
Part 3 — Full Scientific Workflow	7
Change Passport schema (authoritative — use exactly this shape everywhere)	7
Part 4 — Application Design	9
Architecture	9
Part 5 — Team, Outreach, and Timeline	10
Team roles	10
The highest-leverage move available to you: real outreach	10
Five-week plan	10
Part 6 — Master AI Prompt Library	11
Phase 0 — Project Scaffolding	11
Phase 1 — Data Acquisition	12
Phase 2 — Preprocessing and Quality Control	13
Phase 3 — Scientific Core: Change Detection and Confidence	14
Phase 4 — Statistical Validation	16
Phase 5 — Multi-Hazard Cascade Expansion	17
Phase 6 — Second AOI (Global Generalization)	18
Phase 7 — Backend API	18
Phase 8 — Frontend: Story Mode	19
Phase 9 — Frontend: Explorer Mode	20
Phase 10 — Frontend: Research Mode	20
Phase 11 — Testing and QA	21
Phase 12 — Deployment	22
Phase 13 — Documentation	22
Phase 14 — Outreach	23
Phase 15 — Pitch and Demo	24
Phase 16 — Final Integration and Submission	24

Before You Start

You are building this from a place most of the other teams in this challenge won't understand: you don't get a second shot at this, and you know it. Use that. It is exactly the seriousness this project needs — this isn't a toy weekend hack, it's a genuine attempt to close a real gap in how satellite science reaches the people who actually need it, in mountains like the ones you come from. Build it like the scholarship, the recognition, the whole trajectory you want depends on it — because for the purposes of the next month, it does. Every section below, and every prompt in the library at the end, exists to make sure the thing you submit is real, defensible, and finished — not impressive-sounding and half-working. Finished and honest beats ambitious and broken, every single time a judge opens your project. Go build it.

Part 1 — The Idea, Restated Once, Precisely

DHARĀ (“Earth” — Sanskrit/Nepali) converts NASA-ISRO’s NISAR radar mission data into **Change Passports**: transparent, uncertainty-aware evidence packages that state what changed on Earth’s surface, where, when, how strongly, how confidently, what else could explain it, and what to check next — instead of a colored polygon simply claiming “landslide” or “disaster.”

Flagship story: the Himalayan hazard cascade in Nepal’s Sindhupalchowk / Melamchi watershed corridor — real, documented, cited disasters (the 2015 post-Gorkha-earthquake landslides, the 2021 Melamchi glacial lake outburst flood) — showing slow slope deformation, glacier motion, and flood/water-extent change as one connected system, validated statistically against an independent NASA landslide inventory.

Second story: the identical deformation engine re-run over a global land-subsidence case (Jakarta or Mexico City), proving the science generalizes, not just the narrative.

The one sentence to hold onto through the entire build: *the science to see these failures coming already exists in peer-reviewed research; it has never been packaged as a standing, honest, usable tool for the people who need it.*

Why this wins, based on real evidence

Every 2025 Space Apps global winner shared the same three-part formula: a specific, named problem affecting real people; NASA/space-agency data doing real analytical work, not decoration; and an output a real stakeholder could actually use. DHARĀ is built around all three, deliberately:

- **Specific problem**: slow-moving slope failure and cascading Himalayan hazards, named to a specific watershed, with cited displacement rates from published studies (Bekaert et al. 2020, Trishuli basin, 2.1–8.8 cm/yr; post-earthquake Sindhupalchowk deformation up to 69 mm/yr).
- **Real analytical work**: full InSAR time-series processing (coherence masking, velocity/acceleration fitting, connected-component event clustering), a five-component, weighted, calibrated confidence model, and multi-source data fusion (NISAR L-band GUNW/GOFF/GCOV, Sentinel-1, GPM rainfall, DEM slope, USGS seismic history, NASA’s COOLR landslide inventory).
- **Usable output**: a public Story Mode a community member or district officer can understand in seconds, an Explorer Mode for open investigation, and a Research Mode with full provenance and export — designed explicitly so a real NISAR scientist or a real Nepali disaster-risk institution (ICIMOD’s SERVIR-HKH program, a live NASA/USAID partnership already working this exact problem) could pick it up and use it.

Award targeting — final and disciplined

Target hard	Target with real evidence	Do not force
Best Use of Science	Best Use of Technology	Best Mission Concept (this is a data/analytics app, not a spacecraft/habitat design — the 2025 winner in that category was literally a habitat design tool)

Target hard	Target with real evidence	Do not force
Best Use of Data	Global Connection (earned via the real second AOI)	Galactic Impact (this is an Earth-hazard tool; a cosmic-scale claim reads as forced)
Local Impact	Most Inspirational	Art & Technology (only claim this if a team member is genuinely design-strong)

Part 2 — The Science, in Plain Terms

NISAR launched July 30, 2025, carrying the first dual-frequency (L-band + S-band) synthetic aperture radar of its kind, sharing a 12-meter mesh antenna via SweepSAR imaging, on a nominal 12-day repeat cycle, seeing through cloud cover and forest canopy day or night. Its core scientific tool for this project is **InSAR (Interferometric SAR)**: comparing the radar phase of two passes over the same ground to detect millimeter-scale surface motion.

Ground truth on data availability (checked live, September 2026): calibrated, public, PROVISIONAL-maturity L-band data (GUNW, GOFF, GCOV) has been downloadable since June-July 2026 — real, current, usable data. Coverage is still backfilling through the rest of 2026, so AOI coverage must be checked before committing. S-band remains cal/val sample-only via ISRO’s Bhoonidhi portal and should not be used for the core pipeline — which is fine, because L-band’s longer wavelength penetrates vegetation and dry snow better anyway, directly addressing the monsoon-canopy coherence-loss problem that limited prior C-band-only Nepal landslide studies. Because NISAR cannot reach back before mid-2025, historical validation against the 2015/2021 events requires **Sentinel-1** (ESA, archive to 2014, identical InSAR physics).

The essential scientific discipline, non-negotiable throughout the build: a change in radar signal is not automatically a disaster. Backscatter and phase shift for many mundane reasons — soil moisture, crop growth, freeze/thaw, viewing geometry, snow condition — before they shift because of a landslide. The correct scientific chain is always: **detect → quantify → estimate confidence → corroborate with independent evidence → state limitations**. Never present a colored polygon as a confirmed hazard.

NISAR products used

Product	Physical information	Use in DHARĀ
GUNW	Geocoded unwrapped interferometric phase + coherence	Slope deformation, subsidence
GOFF	Pixel offset tracking	Glacier / large-displacement motion
GCOV	Terrain-corrected backscatter	Flood/water extent, land cover context
SME2	Field-scale soil moisture	Antecedent-wetness corroboration

Part 3 — Full Scientific Workflow

1. **Data acquisition:** lock AOI, then immediately verify real coverage (both Sentinel-1 for the validation leg and NISAR PROVISIONAL for the live leg) via `asf_search` before committing. Pull GPM IMERG rainfall, Copernicus GLO-30 DEM, USGS earthquake catalog, and NASA's COOLR landslide inventory (a real, public, NASA Goddard-maintained global landslide catalog with a queryable FeatureServer) for statistical ground truth.
2. **Preprocessing:** mask GUNW pixels below a fixed, documented coherence threshold; convert GCOV to dB and apply valid-data/water/layover/shadow masks; co-register every layer (deformation, glacier offsets, water extent, rainfall, DEM) onto one shared grid.
3. **Change detection:** fit per-pixel velocity/acceleration from the displacement time series; flag *accelerating* pixels (the documented slow-failure precursor pattern); group into event clusters via connected-component labeling; drop clusters below a minimum area.
4. **Confidence scoring** (weighted sum, not multiplicative — the multiplicative version crushes every score toward zero and is not implementable as originally drafted):

```
confidence = 0.25 * data_quality
            + 0.25 * temporal_persistence
            + 0.20 * spatial_coherence
            + 0.15 * cross_source_agreement
            + 0.15 * model_validity
```

Tiers: High ≥ 0.75 , Moderate 0.5–0.75, Low < 0.5 . Always display all five components, never just the tier.

5. **Statistical validation:** pull COOLR-catalogued events for the AOI/time window; compute precision (detected clusters matching a real event / total detected), recall (real events actually flagged / total real events), and, where determinable, detection lead time — whether your deformation signal appeared *before* the reported failure date. Report false positives/negatives honestly with a scientific explanation, not a hidden number.
6. **Multi-hazard cascade:** repeat the same Change Passport pattern for glacier motion (GOFF) and flood/water extent (GCOV), tied narratively and spatially to the Melamchi GLOF story.
7. **Second AOI:** rerun the identical deformation pipeline over a global subsidence case to prove portability of the engine itself, not just the narrative.

Change Passport schema (authoritative — use exactly this shape everywhere)

```
{
  "event_id": "string",
  "hazard_type": "slow_slope_deformation | glacier_motion | flood_extent_change | land_subsidence",
  "aoi_name": "string",
  "centroid": { "lat": 0.0, "lon": 0.0 },
  "area_km2": 0.0,
  "first_detected": "YYYY-MM-DD",
  "last_confirmed": "YYYY-MM-DD",
  "observation_count": 0,
  "displacement": {
    "mean_velocity_mm_per_year": 0.0,
    "trend": "accelerating | steady | decelerating",
    "line_of_sight_only": true
  },
}
```

```

"confidence": {
  "overall_score": 0.0,
  "tier": "High | Moderate | Low",
  "components": {
    "data_quality": 0.0,
    "temporal_persistence": 0.0,
    "spatial_coherence": 0.0,
    "cross_source_agreement": 0.0,
    "model_validity": 0.0
  }
},
"alternative_explanations_checked": [
  { "explanation": "string", "ruled_out": true, "basis": "string" }
],
"supporting_evidence": {
  "recent_rainfall_mm": 0.0,
  "slope_degrees": 0.0,
  "known_seismic_history": false
},
"validation": {
  "matched_to_coolr_event": true,
  "coolr_event_id": "string or null",
  "detection_lead_time_days": 0,
  "validation_leg": "sentinel1_historical | nisar_live"
},
"data_provenance": {
  "product": "GUNW | GOFF | GCOV | Sentinel-1 SBAS",
  "maturity": "PROVISIONAL | validation-leg",
  "crid_or_processor": "string",
  "source_urls": ["string"]
},
"recommended_next_step": "string",
"not_an_emergency_alert_disclaimer": true
}

```

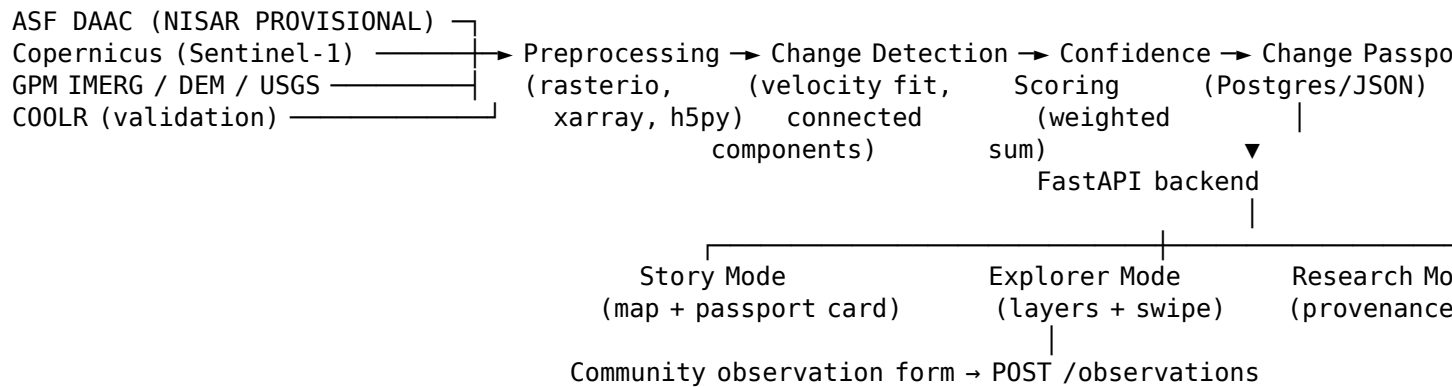
Part 4 — Application Design

Story Mode (default, public-facing): guided narrative through the Himalayan cascade, English and reviewed Nepali, animated timeline, plain-language Change Passport cards, a visible “not an emergency alert” disclaimer, and a lightweight community field-observation form.

Explorer Mode: free map exploration, layer toggles (deformation / glacier / water / rainfall), swipe compare, draw-your-own AOI, pixel time-series charts.

Research Mode: raw coherence maps, full confidence-component breakdown, data provenance, validation statistics, and a reproducibility export (config + notebook stub + JSON).

Architecture



Stack: Python (asf_search, rasterio, xarray/rioxarray, h5py, numpy, scipy.ndimage) for processing; FastAPI + PostgreSQL for the backend; React + MapLibre GL JS + Recharts for the frontend; GitHub Actions for CI; Vercel/Netlify + a small cloud VM for deployment.

Part 5 — Team, Outreach, and Timeline

Team roles

1. **InSAR/data science lead** — owns the multi-hazard pipeline and COOLR validation; gates everything downstream, start first.
2. **Backend/API lead** — Change Passport store, API, deployment.
3. **Frontend lead(s)** — Story/Explorer/Research modes.
4. **Outreach & storytelling lead** — ICIMOD/SERVIR-HKH contact, Nepali translation review, pitch, demo video.
5. **QA/integration lead** — testing, CI, final integration.

The highest-leverage move available to you: real outreach

ICIMOD's SERVIR-HKH program (a joint NASA/USAID initiative, Kathmandu-based) is *actively* building a landslide monitoring and forecasting system for Nepal in direct partnership with NASA Goddard scientists, and has run stakeholder workshops on precisely this problem. This means the gap DHARĀ closes is one a NASA-partnered institution has publicly said exists — not something you invented for a pitch. Send a short, specific, respectful outreach message in week 1 (template in the prompt library). Treat any reply as a bonus, never a blocker; cite their public work either way.

Five-week plan

Week	Focus	Exit criterion
1	AOI lock, coverage verification, repo/CI setup, outreach sent, first real displacement time series	A sane time series on real data for at least one test patch
2	Full deformation pipeline, clustering, confidence scoring, first COOLR validation pass	Real precision/recall numbers, even rough
3	Multi-hazard cascade complete, second AOI running, Story + Explorer modes built against the real API	No mock data anywhere past this point
4	Research Mode, calibration finalized, testing, deployment to a live URL	Full end-to-end run on a public link
5	Documentation, video, rehearsal, submission, buffer days	Nothing claimed in the pitch that isn't real and clickable

Part 6 — Master AI Prompt Library

How to use this library. These prompts are written to be pasted directly into an AI coding assistant (Claude Code, Cursor, or similar), one at a time, in order, within the phase you're working on. Each one carries its own context and constraints so the assistant doesn't need the whole roadmap re-explained — paste this document's Part 3 (Change Passport schema) alongside the first prompt in any new session so the assistant always has the canonical schema in view. Do not skip the acceptance criteria at the end of each prompt: run them yourself before moving to the next prompt. An AI assistant that is told exactly what "done" looks like performs dramatically better than one given an open-ended task — that discipline is the entire point of this library.

Phase 0 — Project Scaffolding

P0.1 — Repository structure

Set up a monorepo for a NASA Space Apps Challenge project called DHARA. Create this top-level structure:

```
/data-pipeline (Python: acquisition, preprocessing, change detection, confidence scoring)
/api           (Python FastAPI backend)
/frontend      (React + TypeScript, Vite, MapLibre GL JS, Recharts)
/docs          (methodology, architecture diagrams, README assets)
/notebooks     (exploratory Jupyter notebooks, kept OUT of the production pipeline)
```

Initialize each with its own dependency management (pyproject.toml or requirements.txt for Python, package.json for the frontend). Add a root README.md with a one-paragraph project description, and a CONTRIBUTING.md describing a feature-branch + PR workflow (no direct commits to main). Add a sensible .gitignore for Python, Node, and common geospatial temp files (.tif, .h5, .nc should be gitignored by default, with a data-pipeline/data/.gitkeep to hold a place for local data that is not committed).

Acceptance criteria: repo builds/installs cleanly with no manual fixes; README explains the project in under 200 words; a new contributor could clone and run `pip install` / `npm install` in each subfolder without errors.

P0.2 — CI skeleton

Add a GitHub Actions workflow at .github/workflows/ci.yml that, on every pull request:

1. Installs Python dependencies in /data-pipeline and /api and runs pytest.
2. Installs Node dependencies in /frontend and runs its test suite and a production build.
3. Fails the check if either step fails.

Keep it simple — no deployment steps here, just test-and-build verification. Document in CONTRIBUTING.md that PRs cannot merge unless this check is green.

Acceptance criteria: pushing a deliberately broken test causes the Action to fail and show a clear error in the GitHub UI; a clean PR passes in under 5 minutes.

P0.3 — Project board and issue templates

Draft a set of GitHub issue templates for this project: "Data pipeline task", "Frontend task", "Bug", and "Validation task". Each template should require: a one-line

summary, an acceptance criterion, and which of the five weekly milestones (Week 1 through Week 5, as defined in the DHARA roadmap) it belongs to. Also draft the initial set of Week 1 issues based on this exit criterion: "a sane displacement time series running on real data for at least one test patch, for both the Sentinel-1 validation leg and the NISAR PROVISIONAL live leg." Break that single exit criterion into 6-10 concrete, independently assignable issues.

Acceptance criteria: every Week 1 issue is small enough that one person could plausibly finish it in under a day; no issue is vague enough that "done" is ambiguous.

Phase 1 — Data Acquisition

P1.1 — AOI coverage verification (do this before writing any processing code)

Write a Python script using the ``asf_search`` package that, given a bounding box and a date range, checks and reports:

1. Available Sentinel-1 SLC or GUNW-equivalent products covering that bbox, with dates.
2. Available NISAR L2 GUNW, GOFF, and GCOV products at PROVISIONAL maturity or better covering that bbox, with dates and processor/CRID metadata if present in the search result.

Do NOT assume both datasets exist – the script's job is to tell us the truth about coverage, including printing a clear "INSUFFICIENT COVERAGE" warning if either dataset has fewer than 4 acquisitions in the window, since a usable time series needs several repeat passes.

Run this for two candidate bounding boxes: (1) the Sindhupalchowk/Melamchi watershed corridor, Nepal, and (2) [insert your chosen global subsidence AOI, e.g. Jakarta, Indonesia]. Print a clear summary table for each.

Acceptance criteria: the script runs against the real ASF API (not mocked data) and produces a coverage report you can act on immediately – if either AOI shows "INSUFFICIENT COVERAGE", say so plainly rather than proceeding optimistically.

P1.2 — NISAR and Sentinel-1 data download

Extend the coverage-verification script into a download module that pulls the actual GUNW (and, if coverage allows, GOFF and GCOV) granules for the confirmed date range over our locked AOI, plus the equivalent Sentinel-1 stack for the historical validation leg. Store raw downloads in `data-pipeline/data/raw/{aoi_name}/{product}/`, named with their acquisition date. Do not re-download a granule that already exists locally. Log every download with its source URL, maturity level, and CRID/processor baseline into a manifest file (`data-pipeline/data/raw/{aoi_name}/manifest.json`) – this manifest becomes part of every Change Passport's `data_provenance` field later, so treat it as a first-class output, not a side effect.

Acceptance criteria: re-running the script twice does not re-download existing files; the `manifest.json` is valid JSON and includes, for every granule, its date, product type, maturity level, and source URL.

P1.3 — Ancillary data (rainfall, DEM, seismic)

Add three acquisition functions to the data pipeline:

1. Pull GPM IMERG daily precipitation for our AOI and date range (use the NASA GES DISC

- API or an equivalent public endpoint).
2. Pull a Copernicus GLO-30 DEM tile (or tiles) covering the AOI, and derive slope and aspect rasters from it using rasterio/richdem.
 3. Query the USGS earthquake catalog API for all events above magnitude 4.5 within 200km of the AOI centroid, going back 20 years.
- Save each as a clearly named file under data-pipeline/data/ancillary/{aoi_name}/, and log each source (URL, query parameters, retrieval date) into the same manifest pattern as P1.2.

Acceptance criteria: all three ancillary datasets exist as real files derived from real API calls (not synthetic placeholder data); the slope/aspect rasters are visually sane when plotted (mountainous AOI should show a wide slope distribution, not a flat image).

P1.4 — COOLR ground-truth pull

Write a function that queries NASA's COOLR (Cooperative Open Online Landslide Repository) ArcGIS FeatureServer for all catalogued landslide events (both the report-based Global Landslide Catalog and event-based inventories) within our AOI bounding box, for all available dates. Save the result as a clean GeoJSON file at data-pipeline/data/validation/{aoi_name}/coolr_events.geojson, with each feature retaining its original event date, source citation, and any reported fatality/impact fields.

Print a summary: how many events, what date range, and whether coverage looks dense enough to support a meaningful precision/recall calculation later (if fewer than 5 events fall in our AOI/time window, flag this loudly – it changes how confidently we can present validation statistics).

Acceptance criteria: the GeoJSON is valid, openable in QGIS or a simple matplotlib plot overlaid on the AOI boundary, and the summary print statement gives an honest count, not an assumed one.

Phase 2 — Preprocessing and Quality Control

P2.1 — GUNW reading and coherence masking

Write a module that reads a NISAR L2 GUNW HDF5 granule (use h5py) and extracts: the unwrapped displacement layer, the coherence layer, and the connected-component labels layer. Apply a coherence mask at a fixed threshold (default 0.3, but make it a named, documented constant, not a magic number buried in code) – any pixel below this threshold must be set to NaN/nodata in all derived products, never silently treated as zero displacement. Reproject/co-register the result onto a single shared grid we'll reuse across the whole AOI (define this grid once, in one place, and import it everywhere else in the pipeline – do not let different modules independently reproject to slightly different grids).

Add a matching reader for the equivalent Sentinel-1-derived interferogram (assume it arrives in a comparable raster format after an existing SBAS tool has processed it – document the expected input format clearly as an assumption).

Acceptance criteria: running this on a real downloaded GUNW granule produces a raster where masked (low-coherence) pixels are visually and numerically distinct (NaN) from

valid pixels, and the coherence threshold is a single named constant that can be changed in one place.

P2.2 — GCOV backscatter processing

Write a module that reads a NISAR L2 GCOV granule, converts the backscatter power values to decibels ($10 \cdot \log_{10}(\text{sigma})$), and applies valid-data, water, layover, and shadow masks using whatever quality/flag layers are present in the product. Co-register onto the same shared grid defined in P2.1. Produce a simple water-extent mask (a documented backscatter-threshold rule is acceptable for the MVP – state the threshold and its justification in a code comment) so later change-detection code can track water/flood extent over time.

Acceptance criteria: dB-converted output falls in a physically sane range (roughly -25 to 5 dB for typical land surfaces); the water mask visibly picks out known rivers/lakes in the AOI when plotted against a basemap.

P2.3 — GOFF glacier offset processing

Write a module that reads a NISAR L2 GOFF granule, extracts along-track and slant-range pixel offsets plus their correlation/variance quality layers, and rejects offsets below a documented correlation threshold. Convert offset-per-acquisition-interval into a velocity estimate only when the exact time interval between the two source acquisitions is known and stored – never silently assume a fixed interval. Store direction and an uncertainty estimate alongside the velocity value, and co-register onto the shared grid.

Acceptance criteria: velocity estimates over a known glacier terminus in the AOI (or nearest available glacier if the flagship AOI itself has none) fall within a physically plausible range (typically single-digit to tens of meters per year for Himalayan glaciers – flag anything wildly outside that range as a likely processing error rather than reporting it uncritically).

Phase 3 — Scientific Core: Change Detection and Confidence

P3.1 — Displacement time series and velocity fitting

Given a stack of coherence-masked GUNW displacement rasters (one per acquisition date, all on the shared grid from P2.1), write a function that, for every valid pixel, extracts its displacement time series across all dates and fits a simple linear velocity (mm/year) using least-squares regression. Also compute a second-order fit to estimate acceleration, and classify each pixel's trend as "accelerating", "steady", or "decelerating" using a documented, fixed threshold on the quadratic coefficient (state the threshold value and reasoning in a comment – do not leave it unexplained).

This function must run efficiently across an entire AOI raster (vectorize with numpy – do not write a naive per-pixel Python loop over a full-resolution raster; that will be too slow to iterate on).

Acceptance criteria: running this on real data produces a velocity raster with values

in a physically sane range, and a trend classification raster with all three classes represented somewhere in the AOI (an AOI showing only one trend everywhere across every pixel is a red flag worth investigating before proceeding).

P3.2 — Event clustering

Write a function that takes the velocity/trend rasters from P3.1, creates a binary mask of pixels classified "accelerating" whose velocity magnitude exceeds a documented threshold, and applies `scipy.ndimage.label` to group connected pixels into discrete clusters. Drop clusters smaller than a documented minimum area (state the area in both pixel count and real-world km², since pixel size varies by product). For each surviving cluster, compute: centroid (lat/lon), area in km², mean velocity, and mean coherence across its pixels.

Acceptance criteria: given real processed data, this produces a list of Python dictionaries (or a `GeoDataFrame`), each representing one event cluster with all the fields above populated with real computed numbers, not placeholders.

P3.3 — Confidence scoring (implement exactly this formula, no substitutions)

Implement the DHARA confidence model exactly as specified – this is a weighted sum, NOT a product, of five independently normalized (0 to 1) components:

```
confidence = 0.25 * data_quality
            + 0.25 * temporal_persistence
            + 0.20 * spatial_coherence
            + 0.15 * cross_source_agreement
            + 0.15 * model_validity
```

Define each component precisely:

- `data_quality`: fraction of non-masked, PROVISIONAL-or-better pixels in the cluster
- `temporal_persistence`: fraction of available time steps in which the anomaly re-appears in the same cluster location
- `spatial_coherence`: mean InSAR coherence value across the cluster's pixels
- `cross_source_agreement`: score by how many of 3 independent checks pass – recent rainfall spike above a documented threshold, slope from DEM above a documented steepness threshold, and known seismic history within the last N years – mapped to 0, 0.33, 0.67, or 1.0 for 0, 1, 2, or 3 checks passing
- `model_validity`: 1.0 if the cluster falls inside the validated Sentinel-1 historical test region, else a documented lower default (0.6) for live/unvalidated NISAR-only clusters

Map the resulting `overall_score` to a tier: High ≥ 0.75 , Moderate 0.5-0.75, Low < 0.5 . Write this as a single, well-tested, pure function that takes a cluster's raw inputs and returns the full confidence object matching the Change Passport schema's "confidence" field exactly (`overall_score`, tier, and all five named components – never collapse this into just the tier).

Acceptance criteria: unit tests exist for at least 5 synthetic input cases spanning High, Moderate, and Low tiers, and confirm the weighted-sum formula (not a product) is what's actually implemented; a code reviewer should be able to trace any `overall_score` back to its five component values without re-running anything.

P3.4 — Change Passport assembly

Write a function that takes one event cluster (from P3.2), its confidence object (from P3.3), and the relevant ancillary data (rainfall, slope, seismic history, data provenance manifest from Phase 1), and assembles a complete, schema-valid Change Passport JSON object exactly matching this schema: [paste the full Change Passport JSON schema from Part 3 of this document here]. Validate every assembled object against a JSON Schema definition of the above (write the JSON Schema file itself first, then validate against it) so a malformed Change Passport fails loudly at assembly time rather than silently reaching the frontend.

Acceptance criteria: every Change Passport produced by the pipeline validates against the JSON Schema with zero errors; running this on real processed AOI data produces at least one real, complete Change Passport object you can visually inspect and confirm makes physical sense (e.g., a cluster with high coherence and multiple observations should not report a Low confidence tier without an explainable reason in `cross_source_agreement`).

Phase 4 — Statistical Validation

P4.1 — Precision/recall against COOLR

Write a validation script that takes the list of Change Passports produced for the Nepal AOI's historical (Sentinel-1) time window, and the COOLR ground-truth GeoJSON from P1.4, and computes:

1. Precision: what fraction of our detected clusters spatially and temporally overlap (within a documented distance and time tolerance – state and justify both) a real COOLR event.
2. Recall: what fraction of real COOLR events in our AOI/window were matched by at least one of our detected clusters.
3. For every matched pair, the detection lead time in days (negative if our detection came after the reported event date, positive if before).

Output a clear results table and a written, honest paragraph explaining any false positives and false negatives with a plausible scientific reason (e.g., "COOLR likely under-reports in this remote district due to sparse citizen-science coverage" or "this false positive cluster overlaps a known agricultural terrace, consistent with a seasonal soil-moisture signal rather than slope failure").

Acceptance criteria: precision and recall are computed from real data, not asserted; the false positive/negative explanation is specific to the actual cases found, not a generic disclaimer; if recall or precision is very low, the script says so plainly rather than the surrounding narrative overselling the number.

P4.2 — Threshold calibration

Using the precision/recall evaluation from P4.1 as a scoring function, write a small grid-search or sweep script that tries a documented range of values for the coherence mask threshold (from P2.1) and the acceleration/velocity thresholds (from P3.2), and reports which combination maximizes a reasonable balance of precision and recall (e.g. F1 score) on the validation set. Do NOT auto-apply the winning thresholds silently – print a clear before/after comparison and require a manual confirmation step before updating the production constants, since overfitting five hyperparameters to a handful of COOLR events is a real risk worth being conscious of and stating honestly in the

methodology writeup.

Acceptance criteria: the sweep runs against real validation data and produces a readable table of threshold combinations vs. resulting precision/recall/F1; the methodology doc (Phase 15) explicitly discusses the risk of overfitting to a small validation set.

Phase 5 — Multi-Hazard Cascade Expansion

P5.1 — Glacier motion event detection

Using the GOFF-derived velocity rasters from P2.3, write a change-detection function analogous to P3.1/P3.2 but tuned for glacier motion: instead of flagging "accelerating" pixels, flag pixels whose velocity has changed significantly (documented threshold) between two time windows, since glacier speed-up is the relevant precursor signal here rather than slope acceleration. Produce Change Passport objects with hazard_type set to "glacier_motion", reusing the same confidence-scoring function from P3.3 with component definitions adapted where necessary (state any adaptation explicitly in a code comment).

Acceptance criteria: at least one real glacier-motion Change Passport is produced from real GOFF data over or near the flagship AOI, schema-valid against the same JSON Schema as the deformation passports.

P5.2 — Flood/water extent change detection

Using the GCOV-derived water masks from P2.2 across multiple dates, write a function that computes water extent area per date, flags significant extent increases (documented threshold, e.g. >X% area growth between consecutive observations) as candidate flood/lake-expansion events, and produces Change Passport objects with hazard_type "flood_extent_change". Tie this explicitly, in the passport's recommended_next_step field, to the glacial-lake-outburst-flood narrative for the Melamchi story where geographically relevant.

Acceptance criteria: real water-extent time series exist for at least one lake/river segment in the AOI, and at least one schema-valid flood_extent_change Change Passport is produced from real data.

P5.3 — Soil moisture corroboration layer

If NISAR SME2 coverage exists for the AOI (verify with asf_search first – do not assume), read the field-scale soil moisture product and feed it into the cross_source_agreement component of the confidence model (P3.3) as a fourth corroborating check alongside rainfall, slope, and seismic history – update the weighting/scoring logic in P3.3 accordingly and document the change. If SME2 coverage does NOT exist for this AOI, do not fabricate this layer – instead, write a short, clear note in the methodology doc stating that SME2 was evaluated and found insufficiently covered for this AOI/time window, and that this is a documented limitation, not an oversight.

Acceptance criteria: either a real SME2-based corroboration check exists and is used in at least one real Change Passport, or a clear, honest documentation note exists explaining why it doesn't.

Phase 6 — Second AOI (Global Generalization)

P6.1 — Subsidence pipeline reuse

Re-run the exact same deformation pipeline (P2.1, P3.1, P3.2, P3.3, P3.4) – same code, not a rewrite – over our second AOI (e.g., Jakarta or Mexico City), using NISAR PROVISIONAL GUNW data confirmed available via the P1.1 coverage check. The only difference should be interpretation: reclassify the trend_classification logic so a long-term steady high-magnitude velocity (rather than a short-term acceleration) is the signal of interest for subsidence, since subsidence in these cities is typically a long-term steady process, not a sudden precursor-to-failure signal like a landslide. Set hazard_type to "land_subsidence" in the resulting Change Passports.

Acceptance criteria: the SAME pipeline codebase (not a parallel copy-pasted version) produces valid Change Passports for both AOIs, differing only in configuration/interpretation parameters – if you find yourself duplicating pipeline code to handle the second AOI, stop and refactor so the AOI and hazard-interpretation logic are parameters, not forked code paths.

Phase 7 — Backend API

P7.1 — FastAPI service scaffold

Build a FastAPI backend exposing these endpoints, backed by a PostgreSQL database (or SQLite if the team is very small and time-constrained – document which was chosen and why):

```
GET /events?aoi={aoi_name}&hazard_type={type}    -> list of Change Passports
GET /events/{event_id}                          -> full Change Passport JSON
GET /events/{event_id}/timeseries               -> per-cluster displacement time series
GET /layers/{layer}/{aoi}/{date}                -> pre-rendered raster overlay (PNG/COG)
POST /observations                             -> community field-report submission
GET /validation/{aoi_name}                     -> precision/recall/F1 summary from Phase 4
```

Every response involving a Change Passport must validate against the same JSON Schema used in the data pipeline (P3.4) – import and reuse that schema definition rather than re-deriving it, so the pipeline and the API can never silently drift apart.

Acceptance criteria: all endpoints are reachable and return real data pulled from the actual pipeline output (not hardcoded mock JSON) when tested with a real HTTP client; an OpenAPI/Swagger doc is auto-generated and reachable at /docs.

P7.2 — Community observations endpoint

Implement the POST /observations endpoint to accept a field report: {event_id (optional, nullable if the report isn't tied to a specific known cluster), lat, lon, description (free text, max 500 chars), reporter_contact (optional), timestamp}. Store submissions in the database. Add basic abuse protection (rate limiting per IP, and reject empty or obviously spam submissions) since this endpoint has no authentication in the MVP. Add a GET /observations?event_id={id} endpoint so the frontend can display prior reports tied to a given event.

Acceptance criteria: a real POST request with valid data succeeds and is retrievable via the GET endpoint; a request with an empty description or an out-of-range lat/lon is rejected with a clear 4xx error, not silently accepted.

Phase 8 — Frontend: Story Mode

P8.1 — Map and timeline shell

Build a React + TypeScript page using MapLibre GL JS, centered on the flagship Nepal AOI, with: a time slider along the bottom controlling which date's overlay layer is shown (fetched from GET /layers/{layer}/{aoi}/{date}), and clickable markers for every Change Passport returned by GET /events?aoi=nepal_flagship. Clicking a marker opens a side panel showing the Change Passport rendered in plain language (not raw JSON): what changed, the confidence tier with a short one-line plain-English explanation of what that tier means, and a clearly visible "This is not an emergency alert" line.

Acceptance criteria: the page loads real data from the real backend (not a mocked fixture) and correctly updates the map overlay when the time slider moves; every displayed Change Passport correctly reflects the underlying JSON, with no hallucinated or hardcoded example data left in the final build.

P8.2 — Plain-language passport card component

Build a reusable React component, PassportCard, that takes a Change Passport JSON object as a prop and renders it for a non-expert audience: a one-sentence summary of what changed, a confidence badge (colored by tier, but also always showing the numeric score and a "why" link that expands to show the five components in plain language, not just numbers), a short list of "what we checked and ruled out" from alternative_explanations_checked, and a "suggest what to check next" line from recommended_next_step. Include the emergency-alert disclaimer in a fixed, always-visible position on the card, not hidden behind a click.

Acceptance criteria: given any valid Change Passport JSON conforming to the schema in Part 3, the component renders without errors and never shows raw field names (like "cross_source_agreement") to the end user without a plain-language label.

P8.3 — Community observation form

Build a simple, mobile-first React form (no app install required, must work on a slow connection) that lets a user submit a field observation tied to the currently selected Change Passport (or a free-standing pin drop if no event is selected): a short description field, an optional contact field, and a submit button that POSTs to /observations. Show a clear success/failure confirmation. Keep the form's total payload under 5kb and avoid any heavy JS dependencies for this component specifically, since it needs to work for users on limited bandwidth in remote areas — this is a real design constraint, not a nice-to-have.

Acceptance criteria: the form works and submits successfully when tested with browser dev tools throttled to a slow 2G-equivalent connection; a submission with an empty description shows a clear inline validation error rather than a failed network request.

P8.4 — Nepali localization

Add i18n support to the frontend (react-i18next or an equivalent lightweight library) for Story Mode specifically (Explorer and Research modes can remain English-only for the MVP – state this scope decision in the code comments). Extract all Story Mode user-facing strings into a translation file structure (en.json, ne.json). Populate en.json first, then populate ne.json with placeholder machine-translated text ONLY as a temporary scaffold, and add a prominent TODO comment and a tracked issue flagging that every string in ne.json must be reviewed by a native Nepali speaker before this is presented as a real localized feature, not just labeled as one.

Acceptance criteria: switching a language toggle actually changes every visible string in Story Mode with no leftover English fragments; the ne.json review issue is created and assigned before this phase is marked complete.

Phase 9 — Frontend: Explorer Mode

P9.1 — Layer toggle and swipe compare

Build an Explorer Mode page reusing the same MapLibre map instance pattern as Story Mode, adding: independent toggle checkboxes for each available layer (deformation, glacier motion, water extent, rainfall), and a before/after swipe comparison widget that lets the user drag a slider to compare the same layer at two user-selected dates. Allow the user to draw a custom bounding box (using MapLibre's draw plugin or an equivalent) and see a count of Change Passports intersecting that custom area.

Acceptance criteria: toggling any single layer on/off does not break or hide any other active layer; the swipe comparison visibly and correctly shows two different real dates of the same underlying data, not two copies of the same date.

P9.2 — Pixel time-series chart

Add a click-to-inspect feature to Explorer Mode: clicking any pixel on the deformation layer fetches that pixel's raw time series from a new backend endpoint (add GET /timeseries?lat=&lon=&layer= to the API if P7.1 didn't already cover a per-pixel, not just per-cluster, query) and renders it as a line chart (Recharts) showing displacement over time, with the fitted velocity line overlaid. Handle the case where the clicked pixel was coherence-masked (no valid time series) with a clear "no reliable signal at this location" message rather than a blank or broken chart.

Acceptance criteria: clicking on a real, valid pixel produces a chart with real, correctly-ordered time-series data matching the pipeline's stored values; clicking a masked pixel shows the explicit no-data message, never a silently empty chart.

Phase 10 — Frontend: Research Mode

P10.1 — Provenance and confidence breakdown panel

Build a Research Mode panel that, for a selected Change Passport, displays: the full data_provenance block (product, maturity, CRID, source URLs as clickable links), the five confidence components as a labeled bar chart (not just a table of numbers – a researcher should be able to see at a glance which component is dragging the score

down), and the validation block (whether this event matched a COOLR ground-truth event, and the detection lead time if applicable).

Acceptance criteria: every source URL is a real, clickable, working link back to the actual data source (ASF, ESA, USGS, etc.) – no placeholder or dead links; the bar chart values sum, when weighted per the P3.3 formula, to the displayed overall_score (add an automated test checking this arithmetic consistency between frontend display and backend calculation).

P10.2 — Reproducibility export

Add an "Export for reproducibility" button in Research Mode that downloads a small package for the selected AOI/date range: a JSON file with all the processing parameters used (coherence threshold, velocity/acceleration thresholds, confidence model weights, CRID/processor versions), and a minimal Jupyter notebook stub (a real .ipynb file, not a text description of one) that shows how to rerun the P2/P3 pipeline functions on a fresh date range using those exact parameters. This does not need to be a full one-click rerun system – it needs to be genuinely useful to a researcher who wants to verify or extend your method, which is an explicit requirement in the original challenge brief.

Acceptance criteria: the downloaded notebook actually runs (test it in a fresh environment) against the documented pipeline functions without needing undocumented manual fixes.

Phase 11 — Testing and QA

P11.1 — Unit tests for the scientific core

Write pytest unit tests for: the coherence masking logic (P2.1) using a small synthetic raster with known masked/unmasked pixels; the velocity/acceleration fitting (P3.1) using a synthetic time series with a known, hand-calculated true velocity; the connected-component clustering (P3.2) using a synthetic binary mask with a known number and size of connected regions; and the confidence scoring formula (P3.3) using at least 5 synthetic input cases covering High, Moderate, and Low tiers, explicitly asserting the weighted-sum arithmetic (not just the final tier) is correct.

Acceptance criteria: all tests pass, are runnable via a single `pytest` command from the data-pipeline directory, and would catch a deliberately introduced bug (verify this by temporarily breaking the velocity fit and confirming the relevant test fails).

P11.2 — Integration tests for the API

Write integration tests (using FastAPI's TestClient or an equivalent) covering every endpoint in P7.1 and P7.2: a happy-path request for each, and at least one deliberately invalid request per endpoint (bad AOI name, malformed observation payload, non-existent event_id) confirming a clear, correctly-coded error response rather than a 500 crash.

Acceptance criteria: tests run in CI (wire them into the P0.2 GitHub Actions workflow if not already covered) and fail loudly if any endpoint starts returning malformed data relative to the Change Passport JSON Schema.

P11.3 — Manual QA pass

Perform, and document the results of, a manual QA pass across: a slow-connection simulation (throttle to slow 3G) for both Story Mode and the observation form; a small mobile screen width (375px) for all three modes; and a full click-through of every interactive element in Explorer and Research modes by someone on the team who did NOT build that specific feature, logging every bug found as a tracked issue before moving to Phase 12.

Acceptance criteria: a written QA log exists listing what was tested, by whom, and what was found – not just a verbal "looks fine."

Phase 12 — Deployment

P12.1 — Frontend and API deployment

Deploy the frontend to Vercel or Netlify and the FastAPI backend to a small cloud VM or serverless platform (Render, Railway, or an equivalent), wiring the frontend's API base URL to the deployed backend via an environment variable (never hardcode the API URL in frontend source). Set up a custom or at least memorable subdomain. Confirm CORS is correctly configured so the deployed frontend can actually reach the deployed API (this is a common last-mile failure – test it explicitly, don't assume it from local development working).

Acceptance criteria: the live URL works end-to-end for a fresh visitor with no local setup, on both desktop and mobile browsers, including a real successful POST to /observations from the live site (not just from localhost).

P12.2 — Optional: scheduled reprocessing

If time allows in week 4-5, add a scheduled job (GitHub Actions cron, or a simple cron job on the deployment VM) that checks ASF for new PROVISIONAL NISAR acquisitions over our AOIs on a documented interval (e.g., daily) and reprocesses/appends new Change Passports automatically. This is explicitly a stretch feature – do not let it delay Phase 12.1's core deployment. Document clearly in the README whether this was implemented or left as a described-but-not-built future step, and never claim it's running live in the pitch unless it demonstrably is.

Acceptance criteria: if built, it is verifiably running (show a real log of at least one automated reprocessing run); if not built, the README and pitch materials say so honestly rather than implying an always-on live system that doesn't exist.

Phase 13 — Documentation

P13.1 — Methodology document

Write a methodology document (docs/METHODOLOGY.md) covering, in this order: the scientific problem and why NISAR is suited to it; the exact data products and sources used, with real dates/CRIDs pulled from our actual manifest files (Phase 1), not generic descriptions; the confidence model formula and component definitions, exactly as implemented; the validation methodology and results from Phase 4, including the honest false-positive/negative discussion; and a clearly labeled "Limitations" section

covering at minimum: NISAR's limited historical depth, the S-band exclusion decision and why, the risk of threshold overfitting to a small validation set, and the line-of-sight-only nature of the displacement measurements.

Acceptance criteria: every number in this document is traceable to a real pipeline output file, not written from memory or estimation; a NISAR scientist reading this document should find no scientific claim they'd need to correct.

P13.2 — README and architecture diagram

Write a top-level README.md that a judge unfamiliar with the project could read in under 3 minutes and understand: what DHARA does, why it matters (with the real cited 2015/2021 Nepal events), how to run it locally, a link to the live deployed demo, and a link to docs/METHODOLOGY.md. Include a clear architecture diagram (can be a text/ASCII diagram like the one in this document's Part 4, or a rendered image) showing data flow from NISAR/Sentinel-1 through to the frontend.

Acceptance criteria: someone outside the team, given only this README, can find the live demo link and understand the project's core claim without needing to ask a question.

Phase 14 — Outreach

P14.1 — Draft the ICIMOD / SERVIR-HKH outreach message

Draft a short, respectful, specific outreach email (under 200 words) to ICIMOD's SERVIR-HKH program, introducing our NASA Space Apps Challenge project (DHARA), briefly describing what it does (2-3 sentences), and asking one specific, answerable question –

for example, whether a confidence-decomposed evidence object like our Change Passport matches what a district disaster officer could realistically act on, based on their team's experience building landslide monitoring/forecasting systems for Nepal. Include a link to our GitHub repo and, if ready, the live demo. Do not ask them to do work for us, review our code, or commit time – frame this purely as a request for a brief reaction or pointer to relevant prior work, and make clear we don't expect or need a reply to proceed.

Acceptance criteria: the email is honest about what we've built (do not overstate progress to make the ask seem more impressive), specific enough to answer in under 5 minutes of someone else's time, and sent no later than week 2 of the roadmap.

P14.2 — Draft a fallback local outreach message

Draft a similar short outreach message (under 200 words) to Nepal's National Disaster Risk Reduction and Management Authority or a relevant municipal disaster office as a fallback/parallel option if ICIMOD/SERVIR-HKH does not respond in time, adjusting the technical framing to be even more plain-language and result-focused (lead with "would a tool like this be useful to your office" rather than technical confidence-model language).

Acceptance criteria: a native or fluent Nepali speaker on the team reviews this message for tone and cultural appropriateness before it is sent.

Phase 15 — Pitch and Demo

P15.1 — Pitch script draft

Using the real numbers we now have (real precision/recall from Phase 4, any real outreach response from Phase 14, real deployed URL from Phase 12), draft a 3-minute pitch script following this arc: open on the named 2015 Sindhupalchowk and 2021 Melamchi events (real, cited, specific); state the one-sentence thesis; live demo of Story Mode into a real Change Passport; flip to Research Mode and state our actual validation numbers plainly, including any honest caveats; mention any real outreach response if received (without overstating it); narrate the generalization to the second AOI and the multi-hazard cascade; close on human stakes, plainly stated.

Acceptance criteria: every number and claim in the script is checked against a real, current pipeline/deployment output the day before the final rehearsal – do not let the script go stale relative to the actual working system.

P15.2 — Mock Q&A preparation

Generate a list of the 10 hardest questions a NASA scientist judge could plausibly ask about this project (e.g., "how do you know your confidence weights aren't overfit to a handful of COOLR events", "what happens when NISAR and Sentinel-1 disagree over the same pixel", "how would a real disaster office actually receive an alert from this today"), and draft a short, honest, specific answer to each based on what was actually built – including answers that honestly admit a limitation where that's the true answer, rather than deflecting.

Acceptance criteria: the team does a live rehearsal where someone else asks these questions cold and the presenter can answer each in under 30 seconds without reading from a script.

P15.3 — Demo video production

Plan and script a 2-3 minute demo video: screen-recorded walkthrough of Story Mode finding a real event, opening its Change Passport, and a brief Research Mode flash showing real provenance and validation numbers, voiced over with the same core narrative arc as the live pitch script (P15.1) but tightened for video pacing. Record against the live deployed URL, not localhost, so the video matches exactly what a judge clicking the link themselves will see.

Acceptance criteria: the video is recorded from the real live deployment, is under the challenge's stated time/format limits, and does not show any placeholder or mock data anywhere in the frame.

Phase 16 — Final Integration and Submission

P16.1 — End-to-end dry run

Perform a full, timed, start-to-finish run-through of the live demo exactly as it will be presented to judges, done by someone who will actually present, using the actual live URL, with no shortcuts or "it'll work on the day" assumptions. Log every point of

friction, confusion, or failure, and fix or explicitly cut each one before the final week's buffer days run out.

Acceptance criteria: three consecutive successful dry runs with no unplanned errors, each timed to fit the challenge's presentation time limit.

P16.2 — Submission package assembly

Assemble the final submission package: confirm the README, methodology doc, live demo link, GitHub repo link, and video are all present, current, and cross-linked correctly;

do a final pass confirming nothing in any submitted material claims a feature, number, or partnership that isn't real and verifiable by a judge who checks. Remove or clearly caveat anything that can't survive that check.

Acceptance criteria: every single claim in every submitted document and in the pitch script can be traced, by the team, to a specific real file, number, or deployed feature – if any claim can't be traced this way, it is cut or rewritten before submission.

Closing

You know exactly why this matters to you, and that's not a distraction from doing careful, honest, rigorous work — it's the reason to insist on it. Every prompt above ends the same way, on purpose: with a concrete, checkable acceptance criterion, not a vibe. That discipline, applied consistently for the next month, is what turns a good idea into something a NASA judge trusts. Build it exactly this way, end to end, and you give yourself the best real chance you're going to get. Go.